Smalltalk — Reference

A **Smalltalk** subset compiled with `lex` + `yacc` + `cc` (`smalltalk.l` / `smalltalk.y` lower Smalltalk to C; `cc` lowers the C to .NET IL). The defining idea of Smalltalk is *everything is an object and computation is sending messages*. This implementation keeps that model literally: every value is a boxed object and every operation — even `+` — is a message dispatched at run time by a `send()` function. User classes get a runtime class id; their methods are dispatched by class id + selector, so a C# program can create objects and send them messages too (Activity 9).

```
smalltalk prog.st               # compile to prog.exe (native .NET executable)
smalltalk prog.st -o app.exe    # choose the output name
smalltalk lib.st --dll          # compile classes to a library for C#/VB.NET
```

Quick Reference

```
"comments use double quotes"          'strings use single quotes'    #symbol
| x y |                               "temporary variable declaration"
x := 6.                               "assignment; statements end with ."
x printNl.                            "unary message"
(3 + 4) printNl.                      "binary message (no precedence: left to right)"
arr at: 1 put: 9.                     "keyword message: selector is at:put:"
(x > 0) ifTrue: [ ... ] ifFalse: [ ... ].
[ x < 10 ] whileTrue: [ x := x + 1 ].
1 to: 10 do: [ :i | sum := sum + i ].
5 timesRepeat: [ 'hi' printNl ].

Object subclass: Counter [          "class definition (GNU-Smalltalk style)"
   | count |                        "instance variables"
   init      [ count := 0 ]         "unary method"
   add: n    [ count := count + n ] "keyword method, parameter n"
   count     [ ^ count ]            "^ returns a value"
]
```

Data types

Every value is an **object**: small integers, booleans (`true` / `false`), `nil`, strings (`' ... '`), symbols (`#name`), and instances of user-defined classes. There is no separate "primitive" world — `3` is an object that understands `+`, `factorial`, `printNl`, etc. Numbers here are 32-bit integers.

Statements / Commands

Smalltalk has almost no statement syntax — control flow *is* message sending. The forms are: assignment (`var := expr`), the `^expr` method return, and expression statements (a chain of message sends ending in `.`). Conditionals and loops are messages (`ifTrue:ifFalse:`, `whileTrue:`, `to:do:`, `timesRepeat:`) that this compiler recognises and **inlines** for speed.

Message precedence

The one rule that surprises newcomers: **unary > binary > keyword**, and binary messages have **no** arithmetic precedence — they run strictly left to right. So `3 + 4 * 2` evaluates as `(3 + 4) * 2 = 14`, and `2 + 3 factorial` is `2 + (3 factorial) = 8` because unary `factorial` binds tighter than binary `+`. Parenthesise when in doubt.

Functions

There are no free functions — only **methods** on classes. A method is a *message pattern* (`increment`, `+ other`, or `at: i put: v`) followed by a body in brackets. `self` is the receiver; `^expr` returns a value (a method with no `^` returns `self`). Each method compiles to a C function dispatched by the receiver's class id and the selector string.

Input / Output

`anObject printNl` / `displayNl` prints the object's textual form followed by a newline (and returns the object); `print` / `display` omit the newline. `printString` / `asString` give the text as a string object.

Graphics

None. Smalltalk-80's strength is its object model and live environment; Activity 8 draws a text triangle with nested `to:do:` loops and string concatenation.

Notes

- Comments are in **double quotes**; strings are in **single quotes** (double a quote to embed it: `'it''s'`).
- Assignment is `:=`; equality is `=`; `~=` is "not equal".
- Binary selectors are runs of operator characters: `+ - * / // \\ < > ≤ ≥ = ~= ,` and `,` concatenates strings.
- Write assignment with a space (`x := 5`), so the scanner doesn't read `x:` as a keyword.

Subset boundaries

A faithful core of the language. Not included: the collection classes (`Array` , `OrderedCollection` , `Dictionary`) and their `do:` / `collect:` / `inject:into:` protocol; blocks as first-class values (blocks are supported only as the inlined arguments of the control-flow messages — they can't yet be stored in variables or passed to user methods); class-side methods and class variables; inheritance beyond a single level (every class is effectively a root object); `become:` , metaclasses, and the reflective/live-image facilities; cascades (`obj msg1; msg2`); and non-local return from nested blocks.

Tutorial

Every example was compiled and run with `smalltalk` ; the output shown is real.

1. Your first program

```
'Hello from Smalltalk' printNl.
```

```
Hello from Smalltalk
```

The string literal `'Hello from Smalltalk'` is an object; `printNl` is a unary message asking it to print itself.

2. Objects and messages

```
| x |
x := 6.
(x * 7) printNl.
(3 + 4 * 2) printNl.
5 factorial printNl.
```

```
42
14
120
```

`(3 + 4 * 2)` is `14` , not `11` — binary messages have no precedence and run left to right. `5 factorial` is a unary message; unary binds tightest, so no parentheses are needed.

3. Flow control

Conditionals and loops are messages. `ifTrue:ifFalse:` is sent to a boolean with two blocks; `whileTrue:` is sent to a block:

```
| sum |
sum := 0.
1 to: 10 do: [:i | sum := sum + i].
sum printNl.
(sum > 50) ifTrue: ['big sum' printNl] ifFalse: ['small sum' printNl].

| n |
n := 1.
[n < 5] whileTrue: [n := n * 2].
n printNl.
```

```
55
big sum
8
```

4. Iteration

`to:do:` counts with an index; `timesRepeat:` just repeats:

```
1 to: 5 do: [:n | n odd ifTrue: ['odd' printNl] ifFalse: ['even' printNl]].
3 timesRepeat: ['tick' printNl].
```

```
odd
even
odd
even
odd
tick
tick
tick
```

The `[:n | ...]` block names the loop index; `odd` is a unary message every integer understands.

5. Classes and methods

This is the heart of Smalltalk — define a class with instance variables and methods, make instances, and send them messages:

```
Object subclass: Account [
   | balance |
   init        [ balance := 0 ]
   deposit: n  [ balance := balance + n ]
   withdraw: n [ (n ≤ balance) ifTrue: [balance := balance - n]
                               ifFalse: ['insufficient funds' printNl] ]
   balance     [ ^ balance ]
]

| a |
a := Account new.
a init.
a deposit: 100.
a withdraw: 30.
a withdraw: 1000.
a balance printNl.
```

```
insufficient funds
70
```

`Account new` makes an instance; `deposit:` and `withdraw:` are keyword methods; inside a method an instance variable like `balance` is just a name, and `^balance` returns it. The guard inside `withdraw:` is itself an `ifTrue:ifFalse:` message.

6. Memory management

Objects live on the managed **.NET heap** and are reclaimed by the garbage collector — there is no `free`:

- `Counter new` allocates an instance with its instance-variable slots set to `nil`.
- Integers and strings are boxed objects created on demand (`mkint`, `mkstr`).
- The runtime never frees anything explicitly; the CLR's GC handles it.

This matches real Smalltalk, where object lifetime is entirely automatic.

7. Strings

Strings are objects; `,` concatenates and `size` measures:

```
| g |
g := 'Small' , 'talk'.
g printNl.
g size printNl.
```

```
Smalltalk
9
```

8. Drawing a picture — a text triangle

```
| s |
1 to: 5 do: [:i |
   s := ''.
   1 to: i do: [:j | s := s , '#'].
   s printNl].
```

```
#
##
###
####
#####
```

The outer `to:do:` chooses the row; the inner one builds a string of `i` hashes by repeated concatenation — a picture drawn entirely with messages.

9. Calling Smalltalk from C# / VB.NET

Compile a file of classes as a library. Because every value is a boxed object and every operation is a `send()`, C# interops at exactly that level: it creates an instance with `mknew`, sends messages with `send()`, and boxes/unboxes integers with `mkint` / `intval`. Selector strings are pushed into the runtime with `CRuntime.InternString`.

```
"lib.st — Counter is the first class, so its runtime class id is 10"
Object subclass: Counter [
   | count |
   init    [ count := 0 ]
   add: n  [ count := count + n ]
   count   [ ^ count ]
]
```

```
smalltalk lib.st --dll        # → stlib.dll
```

```
using CRuntimeLib;                          // for CRuntime.InternString

CProgram.st_boot();                          // initialise nil / true / false
int c = CProgram.mknew(10, 1);               // a Counter (class id 10, one ivar)
CProgram.send(c, CRuntime.InternString("init"), 0, 0);
CProgram.send(c, CRuntime.InternString("add:"), CProgram.mkint(40), 0);
CProgram.send(c, CRuntime.InternString("add:"), CProgram.mkint(2), 0);
int n = CProgram.intval(CProgram.send(c, CRuntime.InternString("count"), 0, 0));
Console.WriteLine($"Counter value = {n}");
```

```
Counter value = 42
```

C# is literally sending Smalltalk messages to a Smalltalk object. The interop is at the object-runtime level (object handles + selector strings) rather than via native signatures — the honest consequence of a language where *everything* is an object. The natural next steps are first-class blocks and the collection protocol (*Subset boundaries*).